

Relatório 4 - Prática de fundamentos de TypeScript

Kaíque Medeiros Lima

1. Descrição da atividade

a. Tipos básicos, listas e tuplas

A atividade reúne exemplos de TypeScript separados em arquivos para trabalhar os principais recursos da linguagem. No `index.ts` aparecem variáveis com `number`, `string` e `boolean`, além de listas que recebem somente valores de um tipo. Isso deixa mais claro o formato esperado dos dados e ajuda a identificar uma atribuição incompatível durante o desenvolvimento.

Também são usadas tuplas, como `[number, string]`, que definem o tipo de cada posição. A diferença pra uma lista comum é que a ordem passa a fazer parte da estrutura. O exemplo de uma lista de tuplas combina essas duas ideias, permitindo guardar vários pares com o mesmo formato. Já o `any` permite trocar o tipo do valor, mas acaba abrindo mão de parte da checagem que o TypeScript oferece.

b. Uniões, enum e afirmações de tipo

Outro ponto dos exercícios é a possibilidade de aceitar mais de um tipo. O `productId` usa `string | number | boolean`, e o tipo `Grade` aceita `number` ou `string`. O símbolo `|` representa uma união, ou seja, o valor pode pertencer a uma das opções definidas. Isso é diferente de uma interseção, que combina as exigências de tipos diferentes.

O `enum` `Direction` dá nomes para valores relacionados, como `Up`, `Down` e `Left`. Também aparece uma afirmação de tipo aplicada ao `productName`. Essa sintaxe informa ao compilador como o valor deve ser tratado, mas não transforma o dado em tempo de execução. Por isso, ela precisa acompanhar o formato real do valor, em vez de servir só para esconder um aviso de tipo.

c. Objetos, tipos e interfaces

No `objects.ts`, os tipos `Order` e `User` organizam os dados de pedidos e usuários. O usuário tem campos como `nome`, `idade` e `e-mail`, uma lista de pedidos e um método `register`. A propriedade `password` aparece com `?`, mostrando que ela é opcional. Com essa estrutura, a gente consegue olhar a definição e entender o que um objeto precisa fornecer.

O arquivo também combina `Author` e `User` com `&`, formando um objeto que precisa atender aos dois tipos. Nas interfaces, o exemplo `UserInterface` define um nome somente para leitura, um `e-mail` e o método `login`. O `readonly` ajuda a impedir a reatribuição daquela propriedade pelo código tipado. Já o uso de `!` em `user.password` informa ao compilador que o valor não seria nulo ou indefinido, mas não faz uma verificação real; como a senha é opcional, esse cuidado continua sendo necessário na lógica.

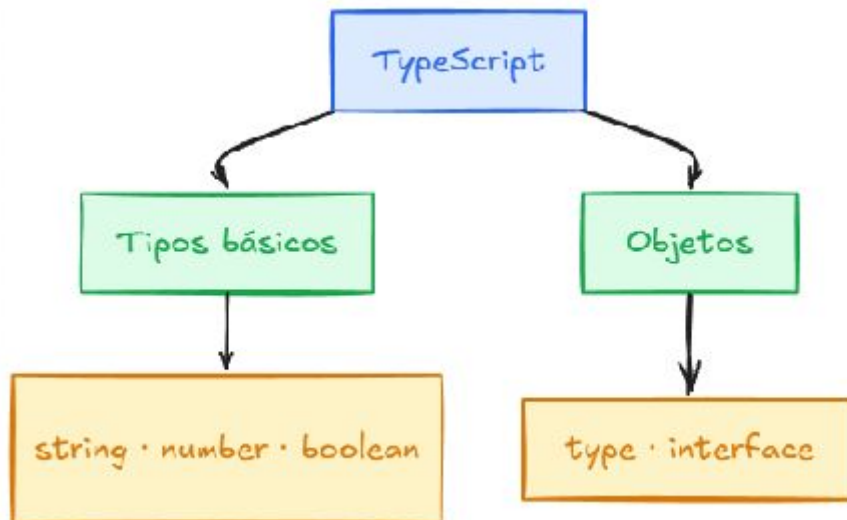


Figura 1 - Tipos básicos e estruturas de objetos. Autoria própria, Excalidraw.

d. Funções e classes

O `functions.ts` define a interface `MathFunc` para funções que recebem dois números e devolvem um número. As funções de soma e subtração seguem esse mesmo formato. Também existe uma função de log com retorno `void`, indicando que ela é usada para executar uma ação, sem devolver um resultado útil para o restante do código.

Na parte de classes, `Person` implementa `IPerson` e reúne identificador, nome, idade e o método `sayMyName`. O `id` é `readonly`, o `nome` é `protected` e a `idade` é `private`. A classe `Employee` herda de `Person` e consegue acessar o nome, mostrando o papel do `protected` na relação entre a classe base e a classe filha. O exemplo `PersonRefact` apresenta uma forma mais curta de declarar essas propriedades diretamente nos parâmetros do construtor.

e. Generics e reaproveitamento

O `generics.ts` mostra como escrever uma mesma estrutura para trabalhar com tipos diferentes. A função `returnValue` usa o parâmetro `T` tanto na entrada quanto na saída, mantendo a relação entre o tipo recebido e o devolvido. Assim, ela pode receber uma string em uma chamada e um número em outra, sem precisar virar uma função baseada em `any`.

O mesmo raciocínio aparece na função que pega o primeiro item de uma lista, na `Promise<string>` e na classe `GenericNumber`. Nessa classe, o valor inicial e a função de soma usam o tipo escolhido na criação do objeto. Isso permite reaproveitar a estrutura e ainda manter um contrato entre os valores. No caso de pegar o primeiro item, a lista vazia continua sendo uma situação que precisa de atenção, porque pode não existir um elemento para retornar.



Figura 2 - Um generic preserva a relação entre entrada e saída. Autoria própria, Excalidraw.

f. Blog “Is TypeScript Frontend or Backend?”

O artigo *Is TypeScript Frontend or Backend?* Apresenta o TypeScript como uma tecnologia que não fica limitada somente ao desenvolvimento frontend. Como ele é construído sobre o JavaScript, pode ser utilizado nos ambientes em que JavaScript é executado, incluindo o Node.js no backend. A principal diferença apresentada é o sistema de tipagem estática, que permite identificar incompatibilidades durante o desenvolvimento e também deixa mais explícito o formato esperado dos dados.

No frontend, o texto mostra o uso do TypeScript junto de ferramentas como Angular, React e Vue. A tipagem pode ser aplicada em propriedades, estados, parâmetros e objetos recebidos pelos componentes. Em vez de depender somente do valor recebido em tempo de execução, parte desses problemas pode ser identificada antes pelo compilador.

O artigo também apresenta o uso do TypeScript no backend, principalmente junto ao Node.js e na construção de APIs. Nesse caso, os tipos podem ser usados para definir parâmetros de funções, objetos recebidos e respostas esperadas.

Também é mostrado que as duas tecnologias não precisam necessariamente competir, já que o TypeScript pode ser usado no frontend enquanto uma API em PHP fica responsável pelo backend.

2. Conclusões

Os exemplos mostram como o TypeScript ajuda a deixar os contratos do código mais visíveis. Tipos básicos organizam os valores, interfaces e tipos descrevem objetos, e os generics permitem reaproveitar funções e classes sem perder essa informação. Também fica clara a diferença entre a checagem durante o desenvolvimento e as verificações que a aplicação precisa fazer quando está rodando. Uma afirmação de tipo pode orientar o compilador, mas não substitui o cuidado com um dado opcional ou com uma lista vazia.

3. Referências

<https://www.youtube.com/watch?v=ppDsxbUNtNQ>

<https://accesto.com/blog/typescript-front-or-back-end/>